THE UNIVERSITY OF WARWICK

# High Performance Computing

# Advanced MPI - 2

Dr Ligang He

# MPI functions

MPI is a complex system comprising of numerous functions with various parameters and variants

Six of them are indispensable, but can write a large number of useful programs already

Other functions add flexibility (datatype), robustness (non-blocking send/receive), efficiency (communication mode), modularity (communicators, groups) or convenience (collective operations, topology).

In the lectures, we are going to cover most commonly encountered functions

# MPI functions

MPI is a complex system comprising of numerous functions with various parameters and variants

Six of them are indispensable, but can write a large number of useful programs already

Other functions add flexibility (datatype), robustness (non-blocking send/receive), efficiency (communication mode), modularity (communicators, groups) or convenience (collective operations, topology).

In the lectures, we are going to cover most commonly encountered functions

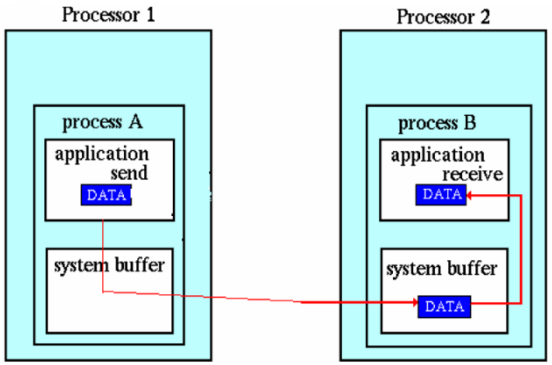
# Communication mode

- The communication mode is only related to the send routines and the receive routine does not specify communication mode

- A communication mode that rule that send operation follows about buffering or synchronistaion.

- There are 4 communication modes
  - Standard mode
  - Synchronous mode
  - Buffered mode
  - Ready mode

# Standard Mode



Path of a message buffered at the receiving process

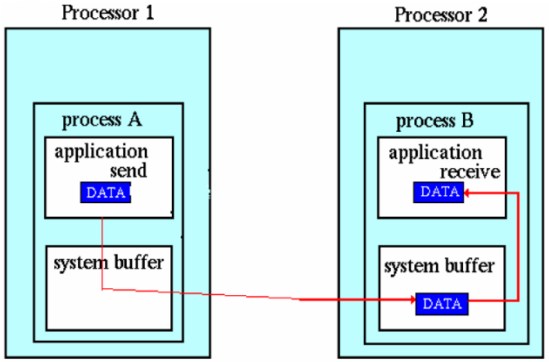
- **The messages are handled in the standard way designed by the MPI implementation**
- **The messages are copied to system buffer (or may not depending on the MPI implementations)**
- **Can have acceptable performance for all possible communication scenarios, but may not give the best performance in certain situations**
- **Blocking Standard send and non-blocking standard send**
  - Blocking send (MPI_Send): the routine returns after the application buffer can be reused
  - Non-blocking send (MPI_Isend): the routine returns immediately

# Standard Mode

- What happens next after the data has been copied to system buffer?
  - At the sender, the MPI system
    - sends a “ready to send” message to the receiver,
    - waits for a “ready to receive” message from the receiver,
    - Starts transferring the data after receiving the “ready to receive”
  - At the receiver, when the receive routine is called, the MPI system sends a “ready to receive” message to the sender
  - This is synchronization: MPI at the sender side does not send data until receiver is ready to receive.

# Standard Mode

- What will happen if the message to be sent is bigger than the system buffer?
  - The send routine will block until
    - 1) the receive routine starts receiving data and therefore empty the system buffer;
    - 2) the rest of the message has been copied to the system buffer.



Path of a message buffered at the receiving process

# Examples of communication scenarios

**Process p0:**

1. Call MPI_Send to send message A to p1;

2. Call MPI_Recv to receive message B to p1;

**Process p1:**

1. Call MPI_Send to send message B to p0;

2. Call MPI_Recv to receive message A to p0;

Case1: The sizes of message A and B are less than their system buffer

Case2: Size of message A exceeds its system buffer - What will happen?

Case3: Sizes of both message A and B exceed their system buffers - What will happen?

# Solve the deadlock in Case 3

## Use non-blocking send

**Process p0:**

1. Call MPI_Isend to send message A to p1;

2. Call MPI_Recv to receive message B to p1;

**Process p1:**

1. Call MPI_Isend to send message B to p0;

2. Call MPI_Recv to receive message A to p0;

**It is the reason why we say non-blocking communication help achieve robustness**

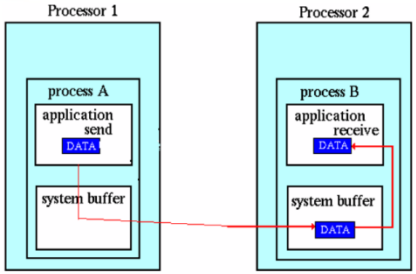
# Synchronous mode (rule about synchronisation)

- **Blocking synchronous send: MPI_Ssend**
  - The send routine blocks until
    - 1) the data has been copied to system buffer and
    - 2) the sender MPI has received the "ready to receive" message
  - If the receive routine is posted later than the send routine,
    - significant synchronization overhead may be incurred

- **Non-blocking synchronous send: MPI_Issend**
  - The routine returns immediately
  - The communication is considered complete after
    - 1) the data has been copied to system buffer and
    - 2) the sender MPI has received the "ready to receive" message

# Buffered mode (rule about buffering)

- The sender uses the user-defined buffer instead of system buffer
  - Needs to make sure the user-defined buffer is big enough
- Communication is considered complete when
  - the application buffer can be reused (data has been copied to the system buffer)
- If the message is bigger than the user-defined buffer, the routine exits (by default).




Path of a message buffered at the receiving process

# Blocking buffered send (1)

- **Format:**

  MPI_Bsend(&buf, count, datatype, dest, tag, comm)

- **The sender doesn't return until the application buffer can be reused**

- **Must attach buffer space using:**

  MPI_Buffer_attach(buffer, size)

- **Buffer space is detached using:**

  MPI_Buffer_detach(buffer, size)

# Blocking buffered send (2)

## Determining the size of the buffer

```
MPI_Buffer_attach(buffer, size );
MPI_Bsend( ..., count=20, datatype=type1, ... );
MPI_Bsend( ..., count=40, datatype=type2, ... );
```

The value of size should be no less than the value computed by

```
MPI_Pack_size( 20, type1, comm, &s1 );
MPI_Pack_size( 40, type2, comm, &s2 );

size = s1 + s2 + 2 * MPI_BSEND_OVERHEAD;
```

MPI_BSEND_OVERHEAD can be found in mpi.h (for C) and mpif.h (for Fortran)

# The deadlock in Case 3

**Process p0:**

1. Call MPI_Send to send message A to p1;

2. Call MPI_Recv to receive message B to p1;

**Process p1:**

1. Call MPI_Send to send message B to p0;

2. Call MPI_Recv to receive message A to p0;

**Use Buffered send to avoid the deadlock**

# Ready Mode (rule about synchronisation)

- In ready mode, sender sends the data straightway
  - Without waiting for the "ready to receive" message
- Programmer must ensure that the receive routine will be called before the send routine
  - Otherwise, ready mode shouldn't be used
- If a ready send is called, but the receive routine has not been called,
  - Error occurs and the routine exits
- Ready mode provides efficiency
  - No need to go through the hand-shaking protocol

# Ready Mode

- Blocking ready send: MPI_Rsend: the sender returns when the application buffer can be reused

- Non-blocking ready send: MPI_Irsend

# Blocking and non-blocking forms for the communication modes

All these four communication modes have both blocking and non-blocking forms

The communication modes refers to the send routines

Standard send: MPI_Send (blocking), MPI_Isend (non-blocking)

Synchronous send: MPI_Ssend (blocking), MPI_Issend (non- blocking)

Buffered send: MPI_Bsend (blocking), MPI_Ibsend (non-blocking)

Ready send: MPI_Rsend (blocking), MPI_Irsend (non-blocking)

# Two Receive routines

**Blocking receive routine: MPI_Recv()**

**Non-blocking receive routine: MPI_Irecv()**

# MPI functions

MPI is a complex system comprising of numerous functions with various parameters and variants

Six of them are indispensable, but can write a large number of useful programs already

Other functions add flexibility (datatype), robustness (non-blocking send/receive), efficiency (ready-mode communication), modularity (communicators, groups) or convenience (collective operations, topology).

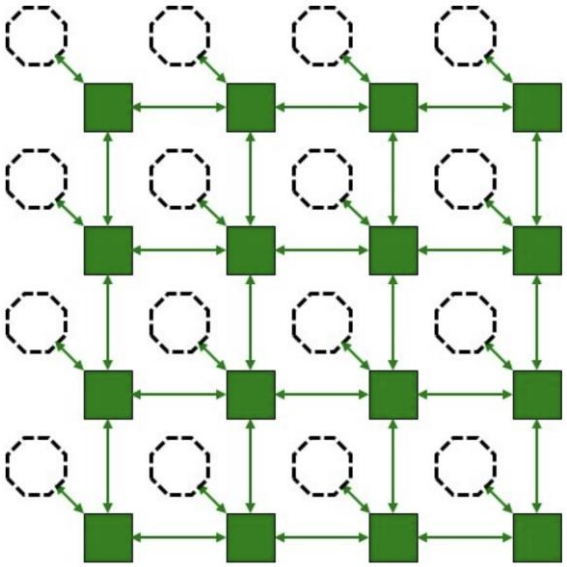
In the lectures, we are going to cover most commonly encountered functions

# Virtual topology

A mechanism for adding an extra indexing system to the processes

To fit the communication pattern better

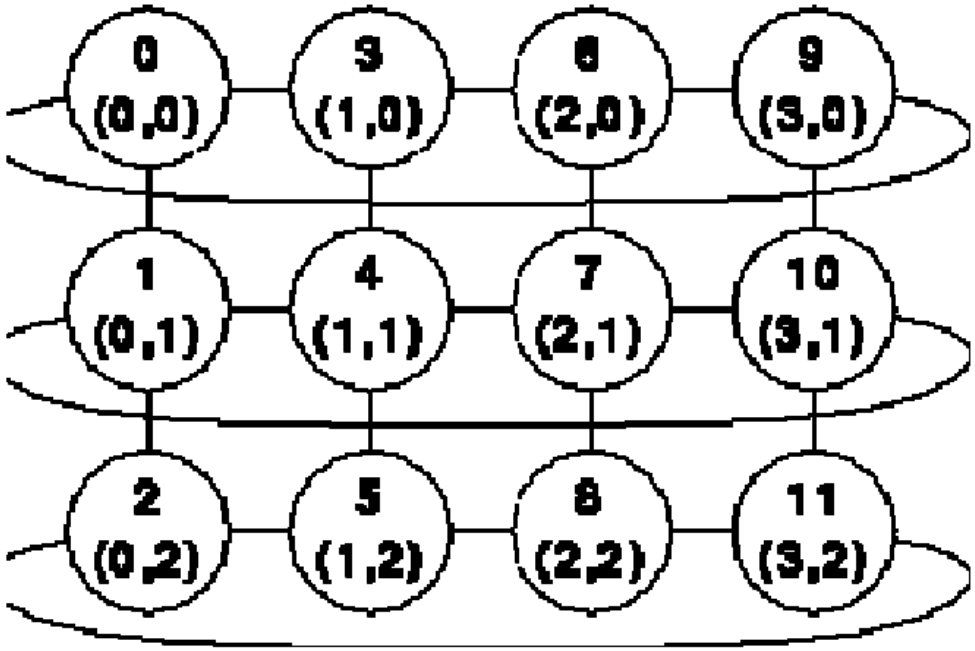
Add convenience to MPI (can make coding easier)



Square – data
Circle – process

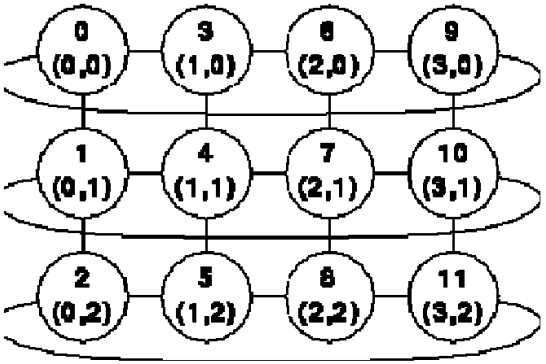
# Cartesian Topology

## Name the processes in a communicator using Cartesian coordinates




Note which is X and Y

# Cartesian topology




- Create a Cartesian topology

- int MPI_Cart_create(MPI_Comm comm_old, int ndims, int *dims, int *periods, int reorder, MPI_Comm *comm_cart)

- A new communicator is created and attaches a coordinate system to processes
  - New communicator defines a mapping between ranks and coordinates
  - The coordinates are only accessible through the new communicator
  - Topology coordinates are extra indices and do not replace ranks

# Converting between ranks and coordinates

MPI_Cart_rank (comm, coords, rank)

converts process grid coordinates to process rank.

It might be used to determine the rank of a particular process whose grid coordinates are known, in order to send a message to it or receive a message from it

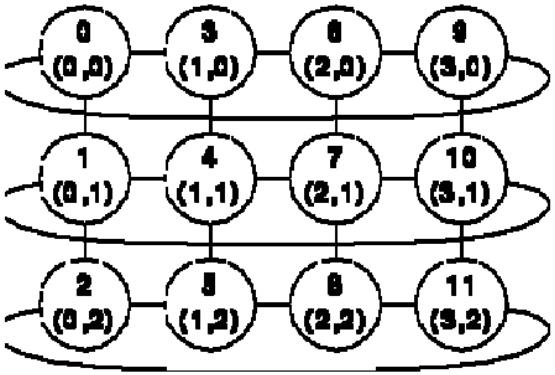
MPI_Cart_coords (comm, rank, ndims, coords)

converts process rank to coordinates.

It might be used to determine the grid coordinates of a particular process from which a message has just been received.

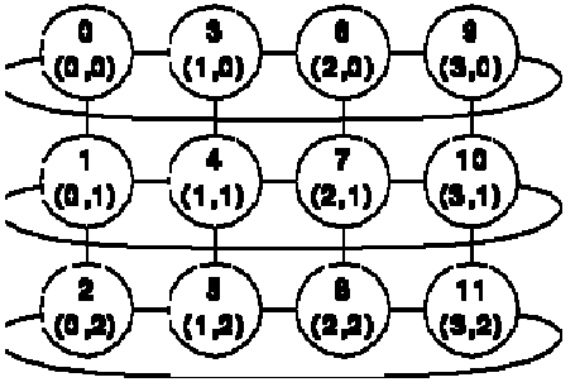
# An Example of Cartesian Topology

```
int main(int argc, char *argv[])
{
   int rank, size;
   MPI_Comm comm;
   int dim[2], period[2], reorder;
   int coord[2], id;
   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   if (size != 12)  {
     printf("Please run with 12 processes.\n");
     MPI_Abort(MPI_COMM_WORLD, 1);
    }

    dim[0]=4; dim[1]=3;
    period[0]=0; period[1]=1;
    reorder=0;
```



# An Example of Cartesian Topology

```
 MPI_Cart_create(MPI_COMM_WORLD, 2, dim,
period, reorder, &comm);
   if (rank == 5) {
      MPI_Cart_coords(comm, rank, 2, coord);
      printf("Rank %d coordinates are %d %d\n",
rank, coord[0], coord[1]);
    }

   if(rank==0) {
       coord[0]=3; coord[1]=1;
       MPI_Cart_rank(comm, coord, &id);
       printf("The processor at position (%d, %d) has
rank %d\n", coord[0], coord[1], id);
    }
    MPI_Finalize();
    return 0
}
```



# MPI functions

MPI is a complex system comprising of numerous functions with various parameters and variants

Six of them are indispensable, but can write a large number of useful programs already

Other functions add flexibility (datatype), robustness (non-blocking send/receive), efficiency (ready-mode communication), modularity (communicators, groups) or convenience (collective operations, topology).

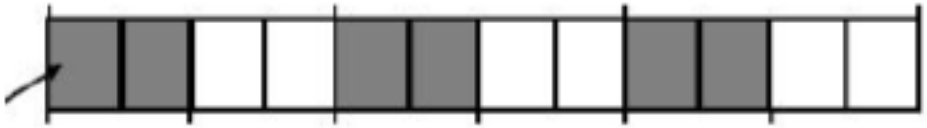
In the lectures, we are going to cover most commonly encountered functions

# Derived datatype

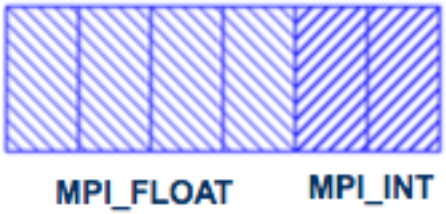
- In MPI_Send, the data to be sent, expressed as a triple(addr, count, datatype), have to occupy contiguous memory address

- Now, there are two problems:

  1. What if the data do not occupy contiguous address



  2. What if the data to be sent do not have the same data type.

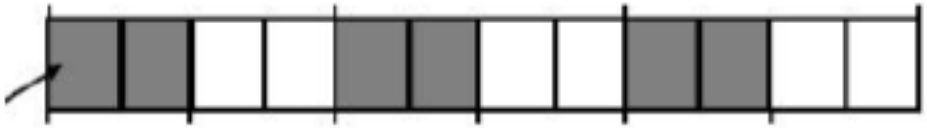


# Derived datatype

- Two possible solutions to problem 1:

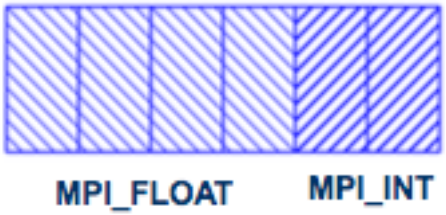
1. Call an MPI_Send for each data block occupying contiguous space.

2. Copy the non-contiguous data to a contiguous buffer space and then call an MPI_Send to send the data in the buffer



- Solution to problem 2:

Call an MPI_Send for each data block which has the same datatype and occupies the contiguous space.




These solutions are tedious and don't have good performance

# Derived datatype

- Derived datatype constructor functions allow the users to construct (derive) their own data types
- Derived datatypes can be used in
  - Grouping data of different datatypes for communication.
  - Grouping non-contiguous data for communication.
- Provides the opportunity to send this kind of data conveniently and potentially improve performance

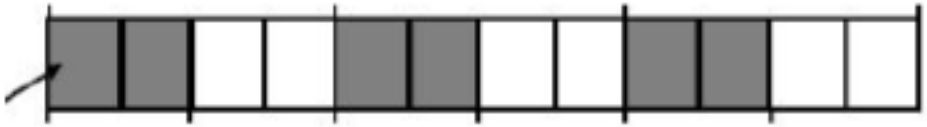
# Data Memory Layout

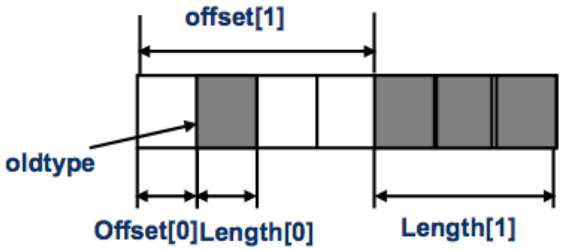
- tell MPI library what bytes in memory belong to the message to be communicated
- is expressed as

{(type_0, offset_0), (type_1, offset_1), ..., (type_n, offset_n)}

- We can construct a new MPI datatype (called derived datatype) for the data based on its memory layout

# MPI Functions for Derived Datatype





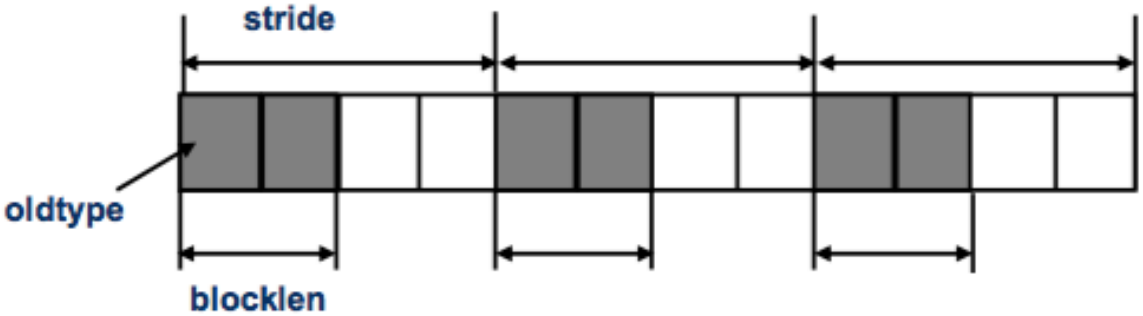

| | Same datatype in each block | Same number of data elements in each block | Same distance between consecutive blocks |
|---|---|---|---|
| MPI_Type_vector | Yes | Yes | Yes |
| MPI_Type_create_indexed_block | Yes | Yes | No |
| MPI_Type_indexed | Yes | No | No |
| MPI_Type_struct | No | No | No |

# MPI_Type_vector

MPI_TYPE_VECTOR (count, blocklen, stride, oldtype, newtype)

Defines a derived type newtype comprising count consecutive blocks of data elements with datatype oldtype, with each block containing blocklen data elements, and the start of successive blocks separated by stride data elements. E.g. (Assume MPI_FLOAT occupies 16 bits)

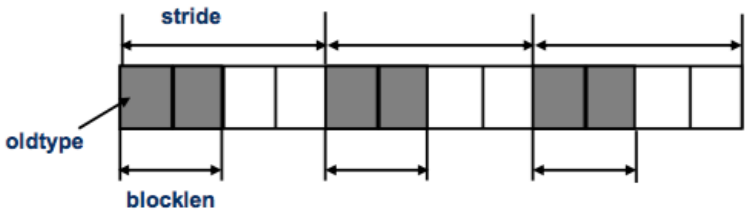
```
float data [1024];

MPI_Datatype floattype;

MPI_TYPE_vector (3, 2, 4, MPI_FLOAT, &floattype);

MPI_Type_commit (&floattype);

MPI_Send (data, 1, floattype, dest, tag, MPI_COMM_WORLD);
MPI_Type_free (&floattype)
```



# MPI_Type_vector

The code in last slide is equivalent to the following code

```
float data[1024], buff[6];
for (i=0; i<3; i++) {
        buff[j] = data [i*4*16];
        buff[j+1]=data[i*4*16+16];
}
MPI_Send (buff, 6, MPI_FLOAT, dest, tag, MPI_COMM_WORLD);
```




Both send 6 FP numbers from locations data[0], data[64], data[128]

→ The memory layout in the example (Assume MPI_FLOAT occupies 16 bits)

- Old datatype = {(MPI_FLOAT, 0)}
- New datatype = {
  (MPI_FLOAT, 0), (MPI_FLOAT, 16),
  (MPI_FLOAT, 64), (MPI_FLOAT, 80),
  (MPI_FLOAT, 128), (MPI_FLOAT, 144) }

| | Same type in each block | Same number of elements in each block | Same distance between consecutive blocks |
|---|---|---|---|
| MPI_Type_vector | Yes | Yes | Yes |
| MPI_Type_create_indexed_block | Yes | Yes | No |
| MPI_Type_indexed | Yes | No | No |
| MPI_Type_struct | No | No | No |

# MPI_Type_create_indexed_block

***int MPI_Type_create_indexed_block( int count, int blocklength, int array_of_displacements[], MPI_Datatype oldtype, MPI_Datatype *newtype );***

***Parameters:***

***count***

- [in] length of array of displacements (integer)

***blocklength***

- [in] size of block (integer)

***array_of_displacements***

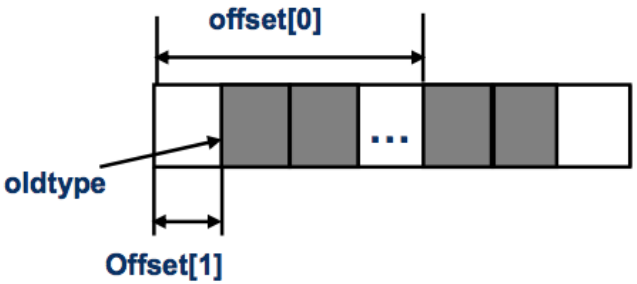
- [in] array of displacements (array of integer)

***oldtype***

- [in] old datatype (handle)

***newtype***

- [out] new datatype (handle)



# MPI_Type_create_indexed_block

*int MPI_Type_create_indexed_block( int count, int blocklength, int array_of_displacements[], MPI_Datatype oldtype, MPI_Datatype *newtype );*

*Let oldtype = {(double, 0), (char, 32)} with extent 40 bits.*

*Let D = (4, 1).*

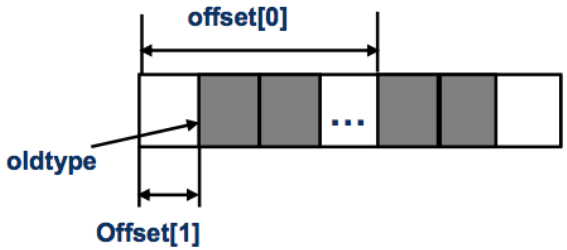
*After calling MPI_Type_create_indexed_block(2, 2, D, oldtype, newtype)*

*newtype= {*

*(double, 160), (char, 192), (double, 200), (char, 232)*

*(double, 40), (char, 72), (double, 80), (char, 112)*

*}*



| | Same type in each block | Same number of elements in each block | Same distance between consecutive block |
|---|---|---|---|
| MPI_Type_vector | Yes | Yes | Yes |
| MPI_Type_create_indexed_block | Yes | Yes | No |
| MPI_Type_indexed | Yes | No | No |
| MPI_Type_struct | No | No | No |

# MPI_Type_Indexed

**MPI_Type_Indexed (count, lengths[], offsets[], oldtype, newtype)**

→**Used in the case where the elements in the datatype to be constructed have the same type, but each block has different number of elements and the distance between two consecutive blocks are different**

→**Used to define a type comprising one or more blocks of a primitive or previously defined datatype, where block lengths and the displacement between blocks are specified in arrays**

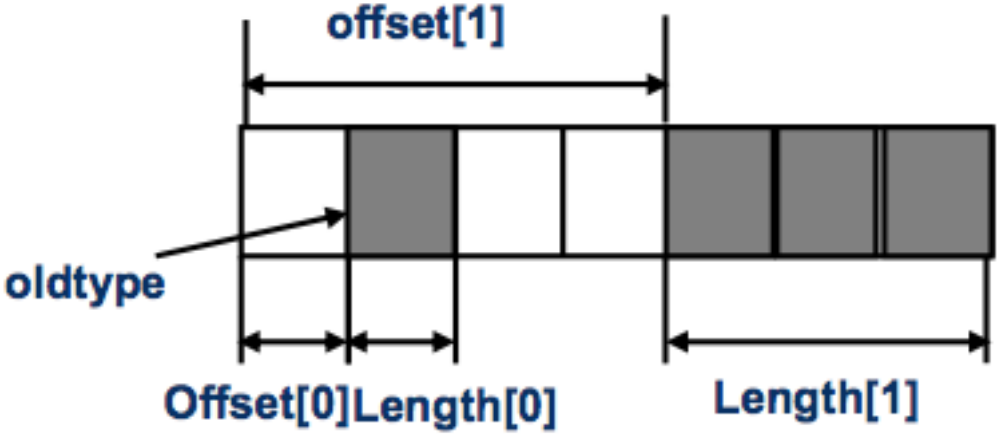
→**The above call defines a type *newtype* comprising count consecutive blocks of data elements with type *oldtype*, with block *i* having a displacement of offsets data elements and containing *lengths* data elements**

# MPI_Type_Indexed

***MPI_Type_Indexed(count, lengths[], offsets[], oldtype, newtype)***

***Let oldtype = {(double, 0), (char, 32)} with extent 40 bits.***
***Let B = (1, 3) and Let D = (4, 1).***
***After calling MPI_Type_indexed(2, B, D, oldtype, newtype)***

*newtype= {*
*(double, 40), (char, 72)*
*(double, 160), (char, 192), (double, 200), (char, 232), , (double, 240), (char, 272)*
*}*



| | Same type in each block | Same number of elements in each block | Same distance between consecutive block |
|---|---|---|---|
| MPI_Type_vector | Yes | Yes | Yes |
| MPI_Type_create_indexed_block | Yes | Yes | No |
| MPI_Type_indexed | Yes | No | No |
| MPI_Type_struct | No | No | No |

# MPI_Type_struct

int MPI_Type_struct(int count, int *array_of_blocklengths, MPI_int *array_of_displacements, MPI_Datatype *array_of_types, MPI_Datatype *newtype)

The derived datatype includes different datatypes, each block with different displacements and different data elements

# MPI_Type_struct

*Let oldtype1 = {(double, 0), (char, 32)} with extent 40 bits, and oldtype2 = {(float, 0), (char, 16)} with extend 24 bits.*

*Let B = (3, 1) and D = (160, 16) and C=(oldtype1, oldtype2).*

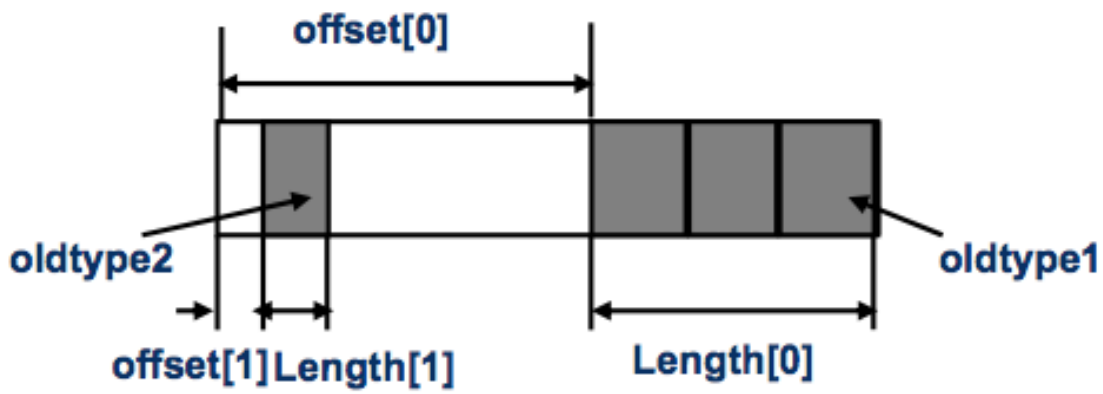
*After calling MPI_Type_struct (2, B, D, C, newtype)*

*newtype= {*

*(double, 160), (char, 192), (double, 200), (char, 232) (double, 240), (char, 272),*

*(float, 16), (char, 32)*

*}*



# Derived datatype summary

| | Same type in each block | Same number of elements in each block | Same distance between consecutive blocks |
|---|---|---|---|
| MPI_Type_vector | Yes | Yes | Yes |
| MPI_Type_create_indexed_block | Yes | Yes | No |
| MPI_Type_indexed | Yes | No | No |
| MPI_Type_struct | No | No | No |

# MPI_Type_contiguous

```
int MPI_Type_contiguous( int count, MPI_Datatype old_type, MPI_Datatype *new_type_p );
```

MPI_Type_contiguous(3, MPI_REAL, newtype)

// returns a new datatype that represents the concatenation of 3 instances of // MPI_REAL.

MPI_Type_commit(newtype)

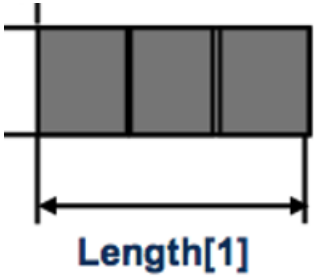
// commits the datatype, must be done before communication

MPI_Send(buff, 1, newtype, dest, tag, MPI_COMM_WORLD)

// sends the data at location data to dest

MPI_Type_free(newtype)

// frees the datatype




This is equivalent to the following single call

MPI_SEND(buff, 3, MPI_REAL, dest, tag, MPI_COMM_WORLD)

Where the elements to be sent are already contiguous

# MPI_Type_contiguous

- **Memory layout in last example(assume MPI_REAL occupies 32 bits)**
  - Old data type = {(MPI_REAL, 0)}
  - New datatype = {(MPI_REAL, 0), (MPI_REAL,32), (MPI_REAL, 64)}

# Summary of MPI

Point-to-point communication

Collective communication

Communication modes

Virtual topology

Derived datatype